

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 11

Query Retrieval Fundamentals

“A retriever can only ever answer the question you actually embedded — not the one you meant to ask.”

Chapter 11 — Query Retrieval Fundamentals

Memora's retriever is deliberately unglamorous. It does one thing — turn a query into ranked evidence — and it does it the same way whether it is called from a simple command-line loop or from deep inside an agent's tool-calling turn. That simplicity is a feature: Chapter 12 will make the ranking smarter, and Chapter 15 onward will make the calling pattern autonomous, but neither upgrade should require touching the retrieval mechanism itself.

11.1 From question to query embedding

Definition — Query embedding

The dense vector produced by running a user's natural-language question through the same embedding model used at ingestion time, so that the question and every stored chunk live in the same geometric space and a distance between them is meaningful.

Retrieval starts with a single call: embed the question. `'RAGRetriever._embed_and_log'` wraps `'EmbeddingManager.generate_embedding'` and immediately logs what came out — the model name, the vector's shape, its first eight values, and its L2 norm. None of that logging changes behavior; it exists because an embedding is otherwise a black box, and the fastest way to catch a wrong model or an empty string is to look at the numbers before they disappear into a similarity search.

```
def _embed_and_log(self, query: str):
    query_embedding = self.embedding_manager.generate_embedding(query)
    _log(
        "STEP: QUERY -> EMBEDDING",
        f'Query          : "{query}"',
        f"Model           : {self.embedding_manager.model_name}",
        f"Shape            : {query_embedding.shape}",
        f"First 8 vals      : {[round(float(v), 6) for v in
query_embedding[:8]]}",
        f"L2 norm         : {float(np.linalg.norm(query_embedding)):.6f}",
    )
    return query_embedding
```

Analogy — A shared coordinate system

Embedding the question and embedding every stored chunk are the same act of translating text onto one map. A distance between two points is only meaningful if both were plotted with the same projection — a query embedded by one model and chunks embedded by another are two maps drawn to different scales, and every distance computed between them is noise.

This is also why the embedding model is never a per-query choice. It is fixed once, at ingestion time (Chapter 8), and the retriever simply reuses it. Swapping embedding models mid-project does not raise an error — ChromaDB will still return a nearest-neighbor list — but the resulting distances stop meaning what the code assumes they mean.

Memora's `'EmbeddingManager'` defaults to `'all-MiniLM-L6-v2'`, a compact SentenceTransformer model chosen for speed over an agentic loop that may embed several query variants per request.

`generate\_embedding` mirrors the retrieval timeout pattern from Section 11.2: `model.encode()` runs inside its own bounded call, and an `EmbeddingEncodingTimeoutError` fires if it exceeds `EMBEDDING\_ENCODING\_TIMEOUT\_SECONDS` rather than letting a stalled encode silently stall the request. The two timeout classes exist for the same reason — a local model call and a network call are both external calls from the pipeline's point of view, and both deserve an upper bound.

11.2 Similarity search mechanics, step by step

With a query vector in hand, `\_query\_collection` submits it to ChromaDB and asks for the closest `n\_results`. Two details keep this simple call from becoming a production liability: the requested count is clamped with `min(top\_k, collection.count())` so a small or freshly created collection never receives an impossible request, and the query itself runs inside a bounded worker so a stalled database connection cannot stall the whole pipeline.

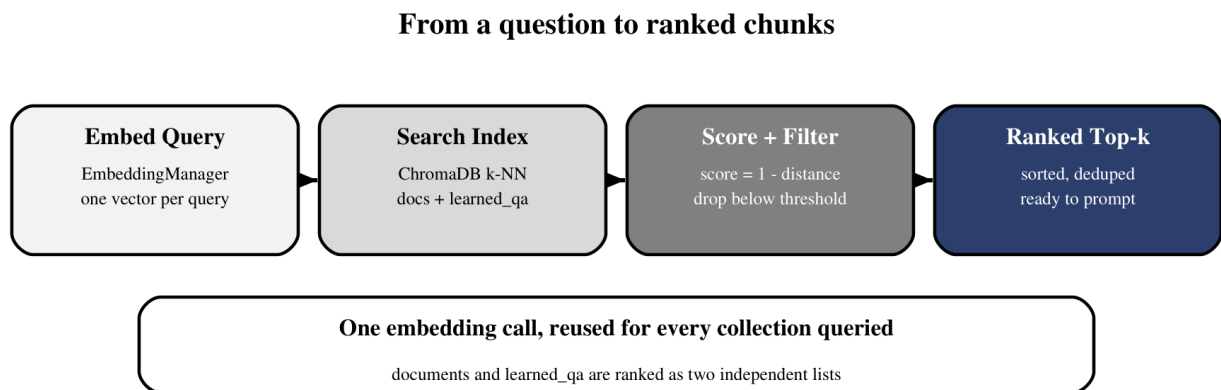


Figure 11.1 — One embedded query drives every collection search that follows.

As Figure 11.1 shows, the same query embedding is reused for every collection queried in a single request — the documents collection and, where present, the learned-QA collection. Each collection returns its own documents, metadatas, and distances, which the retriever immediately reshapes into a list of plain dictionaries carrying an id, content, metadata, similarity score, and raw distance.

```
with concurrent.futures.ThreadPoolExecutor(max_workers=1) as executor:
    future = executor.submit(
        collection.query,
        query_embeddings=[query_embedding.tolist()],
        n_results=min(top_k, collection.count()),
        include=["documents", "metadatas", "distances"],
    )
    try:
        results = future.result(timeout=RETRIEVAL_TIMEOUT_SECONDS)
    except concurrent.futures.TimeoutError:
        raise RetrievalTimeoutError(collection.name,
RETRIEVAL_TIMEOUT_SECONDS)
```

Common pitfall — A vector-store call that never returns

Early versions had no upper bound on a collection query, so a stalled ChromaDB connection could hang the entire request. `'RETRIEVAL_TIMEOUT_SECONDS'` was not guessed once and left alone — it was set from documentation estimates, then recalibrated after a benchmark run measured the actual p95 query latency at roughly 0.7 seconds and fixed the timeout at 10 seconds, about fourteen times that figure. Treat any external call inside a retrieval path as a call that can hang, not merely one that can fail.

11.3 Top-k selection, score thresholds, and MIN_SIMILARITY heuristics

Definition — Score threshold

A minimum similarity value below which a retrieved chunk is discarded before it ever reaches ranking or the prompt, regardless of how it compares to the other results in the batch.

`'RAGRetriever.retrieve'` and `'retrieve_separate'` both accept a `'score_threshold'`, but they default it to `'0.0'` — accept everything the database returns, and let the caller decide what counts as relevant. That is a deliberate separation of mechanism from policy: the retriever's job is to search correctly, not to decide what "good enough" means for a particular caller.

Policy lives one layer up. The agent-facing tool in `'tools.py'` calls `'retrieve_separate'` with `'score_threshold=MIN_SIMILARITY'`, a single project-wide constant set to `'0.5'`. A simple command-line query script can choose a looser threshold, or none at all, without touching the retriever itself.

Caller	top_k / top_l	score_threshold	Intent
<code>'RAGRetriever.retrieve'</code>	caller-supplied, default 5	0.0 (accept all)	Mechanism only
<code>'query.py: generate_answer'</code>	<code>'--top-k'</code> , default 7	not applied	Fast, permissive CLI answer
<code>'query.py: advanced_answer'</code>	<code>'--top-k'</code> , default 7	<code>'--min-score'</code> , default 0.2	Answer with sources + confidence
<code>'tools.py: retrieve_documents'</code>	<code>'RETRIEVAL_TOP_K'</code> / <code>'RETRIEVAL_TOP_L'</code>	<code>'MIN_SIMILARITY'</code> = 0.5	Agent-facing policy

Reading this table top-to-bottom is reading the project's own escalation in caution: the raw mechanism trusts nothing, the simplest script filters nothing, and the surface the language model actually calls filters hardest, because a model that receives a weakly related chunk will often use it anyway.

11.4 Distance-to-score conversion and what scores actually mean

ChromaDB returns distances, not similarities, and distance is the wrong quantity to reason about — smaller is better, thresholds read backwards, and "0.9" looks like a strong match when it might be a weak one. `\_rank\_collection\_results` converts every distance into a similarity score with `similarity\_score = 1 - distance`, sorts descending by that score, drops anything below the threshold, deduplicates by chunk id, and assigns each surviving chunk a 1-based rank.

```
@staticmethod
def _rank_collection_results(results, limit, score_threshold):
    seen_ids = set()
    ranked = []
    for doc in sorted(results, key=lambda d: d["similarity_score"],
reverse=True):
        if doc["id"] in seen_ids:
            continue
        seen_ids.add(doc["id"])
        if doc["similarity_score"] < score_threshold:
            continue
        ranked.append(**doc, "rank": len(ranked) + 1)
        if len(ranked) >= limit:
            break
    return ranked
```

Score range	Interpretation
0.7 – 1.0	Strong topical match
0.4 – 0.7	Related, same domain
0.2 – 0.4	Loosely related — shared vocabulary only
0.0 – 0.2	Effectively unrelated
below 0.0	Rare — usually a sign of a malformed query or a distance-space mismatch

Common pitfall — The `1 - distance` formula only holds for cosine distance

Memora once ran the `documents` collection with cosine distance and the `learned\_qa` collection with L2 distance, because `learned\_qa` was created before its `hnsw:space` was ever specified, and ChromaDB's default is L2. Both collections were still scored as `similarity\_score = 1 - distance`, which is correct only for cosine distance on normalized vectors — on L2 it computes an unrelated quantity. Relevant learned-answer chunks survived their threshold by a hair, or were silently dropped, and nothing raised an exception because the code was syntactically fine; it was just scoring two collections under two different, incompatible definitions of "distance." Fix the distance metric at the single point where a collection is created, and verify it there — never assume every collection you query shares the same geometry.

11.5 Building the RAGRetriever class

`RAGRetriever` wraps a `VectorStore`, an `EmbeddingManager`, and an optional learned-QA collection behind two public methods: `retrieve`, which searches the documents collection alone, and `retrieve_separate`, which searches documents and learned QA as two independent ranked lists. Keeping them independent — rather than merging into one combined ranking — matters because the two collections answer different questions: one holds source material, the other holds the system's own validated prior answers, and Chapter 22C will give them different compression treatment for exactly this reason.

Figure 11.2 traces `retrieve_separate` end to end. The embedding step runs exactly once regardless of how many collections exist; everything after it forks into independent, symmetric tracks that share the same querying, ranking, and filtering logic but never influence each other's scores or thresholds. A learned-QA collection with zero entries — true for every fresh install — simply produces an empty right-hand track rather than a special case the caller must detect.

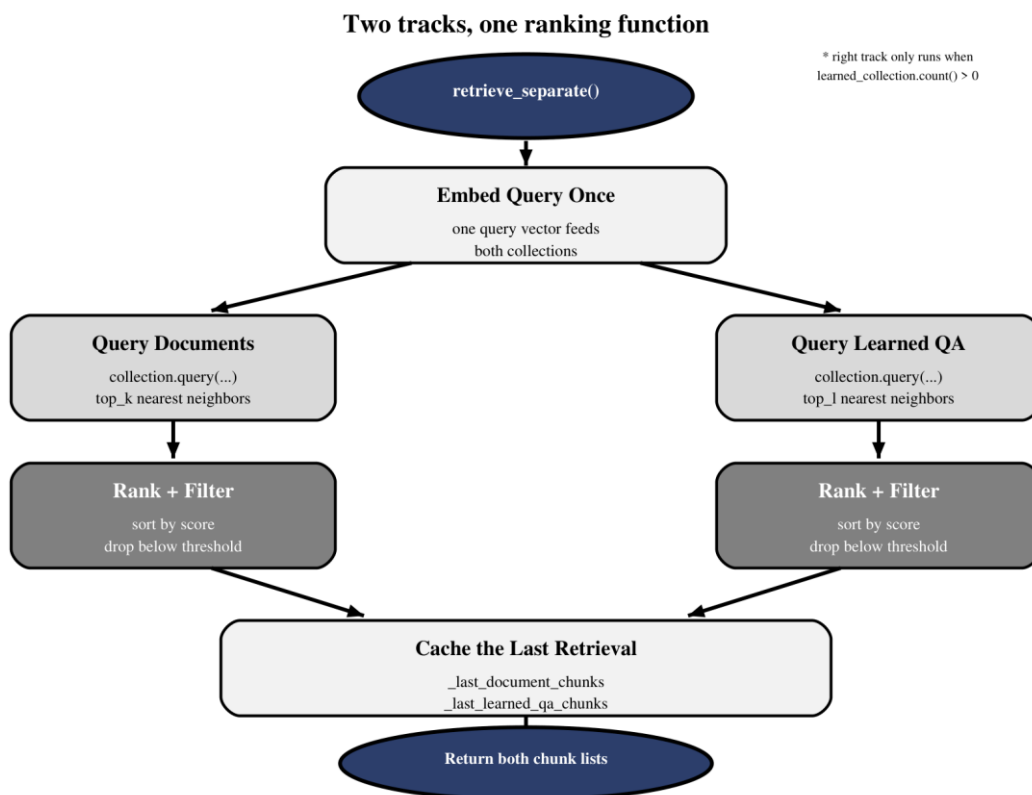


Figure 11.2 — `retrieve_separate()` runs both tracks through the same embed-query-rank pipeline, independently.

```

class RAGRetriever:
    def __init__(self, vector_store, embedding_manager,
learned_collection=None):
        self.vector_store = vector_store
        self.embedding_manager = embedding_manager
        self.learned_collection = learned_collection
        self._last_document_chunks: list[dict] = []
        self._last_learned_qa_chunks: list[dict] = []

    def retrieve_separate(self, query, top_k, top_l, score_threshold=0.0):
        query_embedding = self._embed_and_log(query)

        documents = self._rank_collection_results(
            self._query_collection(self.vector_store.collection,
query_embedding, top_k),
            limit=top_k, score_threshold=score_threshold,
        )

        learned_qa = []
        if self.learned_collection and self.learned_collection.count() > 0:
            learned_qa = self._rank_collection_results(
                self._query_collection(self.learned_collection,
query_embedding, top_l),
                limit=top_l, score_threshold=score_threshold,
            )

        self._last_document_chunks = documents
        self._last_learned_qa_chunks = learned_qa
        return {"documents": documents, "learned_qa": learned_qa}

```

- One embedding call per query — never re-embed for a second collection.
- Guard every collection query with `min(top\_k, collection.count())`.
- Rank, dedupe, and threshold-filter each collection independently.
- Return plain dictionaries, not database objects — the rest of the pipeline should never import a ChromaDB type.
- Skip a collection cleanly (empty list) rather than special-casing its absence downstream.

11.6 Inspecting what came back — scores, sources, previews

A ranked list of dictionaries is only useful if a human — or a log file — can read it. ``_log_ranked_results`` writes one line per surviving chunk: its rank, its score to four decimal places, its source filename, and a 120-character content preview with newlines flattened. That single debug block is usually the fastest way to answer "why did the agent say that?" — before looking at prompts, before looking at the model's output, look at what was actually retrieved.

``query.py``'s ``advanced_answer`` builds a caller-facing version of the same idea. Each source entry carries the filename, page, similarity score, and a 300-character preview, and the whole answer's ``confidence`` is simply the highest similarity score among the retrieved chunks — a cheap, honest signal that a caller can display without asking the language model to grade its own homework.

```
sources = [
    {
        "source": doc["metadata"].get("source", "Unknown"),
        "page": doc["metadata"].get("page", "unknown"),
        "similarity_score": doc["similarity_score"],
        "preview": doc["content"][:300] + ("..." if len(doc["content"]) >
300 else ""),
    }
    for doc in retrieved_docs
]
confidence = max(doc["similarity_score"] for doc in retrieved_docs)
```

Analogy — An itemized receipt, not just a total

An answer without its retrieved sources is a total with no line items — plausible, but unauditible. Attaching score, source, and preview to every chunk that fed the answer turns a single confidence number into something a reader can actually check, chunk by chunk, against the claim it supposedly supports.

Notice that the two previews are sized for different readers: the debug log's 120-character preview is tuned for a developer scanning dozens of lines per request in a terminal, while ``advanced_answer``'s 300-character preview is tuned for an end user reading a handful of source cards. Neither number is arbitrary once you ask who reads it and how many of them they will read in one sitting — a log preview optimizes for scan speed across many entries, a source preview optimizes for enough context to judge one entry on its own.

11.7 Why the retriever caches its last chunks (and who consumes that cache)

``retrieve`` and ``retrieve_separate`` both end by writing their results into ``self._last_document_chunks`` and ``self._last_learned_qa_chunks``, exposed through ``get_last_document_chunks()`` and ``get_last_learned_qa_chunks()``. This looks redundant — the caller already has the return value — until you notice who the caller actually is later in the book.

When ``retrieve_documents`` runs as a tool inside the agent loop (Chapters 17–18), its return value is a flattened string formatted for the language model's context window. That string is the only thing the model ever sees, but the orchestrator around the model needs the original structured chunks — with ids, metadata, and per-chunk scores — to track which sources were used, deduplicate across retrieval rounds, and feed compression. Threading structured objects through a channel designed to carry model-readable text would mean re-parsing the retriever's own formatted output. The cache sidesteps that: the orchestrator calls the tool for the model's benefit, then calls ``get_last_document_chunks()`` for its own, immediately afterward, on the same retriever instance.

Analogy — A carbon-copy receipt

The tool result handed to the model is the customer's copy — readable, final, and not meant to be parsed back apart. The retriever keeps its own carbon copy of the same transaction, in full structured detail, for whoever in the system needs to reconcile the books afterward.

Common pitfall — Trusting the cache across two different queries

The cache holds the **last** retrieval only. If anything in the pipeline calls ``retrieve_separate`` again — a reformulated query, a second collection, a retry — the previous chunks are gone from ``_last_document_chunks`` the moment the new call returns. Read the cache immediately after the call it belongs to, not at some later point in the loop where a second retrieval may have already overwritten it.